

NATIONAL INSTITUTE OF TECHNOLOGY CALICUT
Department of Electronics and Communication Engineering



WINTER 2025–26

EC6131E Reconfigurable System-On-Chip

PROJECT REPORT

MTech – Electronics Design and Technology

**FPGA-Based JPEG-like Image Compression Using
Custom AXI4-Lite DCT IP Core on Zynq SoC**

Submitted By:

Chetankumar Budihal (M250737EC)
Donthula Yugandhar Reddy (M250758EC)
Harsh Songire (M250793EC)
Sai Prakash Reddy Chilpuri (M251003EC)

Contents

1	Objective	2
2	System Architecture	2
2.1	Block Design Overview	2
2.2	System Data Flow	3
3	IP Core Design	3
3.1	AXI4-Lite Register Map	3
3.2	Hardware Pipeline State Machine	3
3.3	DCT Implementation	3
3.4	Quantisation	4
3.5	RLE Encoding	4
4	Design Code	4
4.1	Verilog – AXI4-Lite Slave and DCT Engine (myip_DCT_v1_0_S00_AXI) . .	4
4.2	C – Vitis Application (main.c)	5
4.3	Python – UART Receiver and Image Reconstruction (receive.py)	7
5	Procedure	8
6	Results	8
6.1	UART Terminal Output (Python Host)	8
6.2	Image Compression Output	9
6.3	Compression Statistics	10
6.4	Block-Level RLE Sample	10
7	Conclusion	10

Objective

Design, implement, and demonstrate a custom AXI4-Lite IP core on the Zynq-7000 SoC (ZedBoard) that performs JPEG-like lossy image compression. The IP core implements:

1. Two-pass 2-D 8×8 Forward Discrete Cosine Transform (DCT)
2. Zigzag scan reordering of DCT coefficients
3. JPEG standard luminance quantisation using a reciprocal-multiplication approach
4. Run-Length Encoding (RLE) with End-of-Block (EOB) and Zero-Run-Length (ZRL) markers

The Processing System (PS) feeds pixel blocks to the IP through AXI registers, retrieves RLE-encoded data, and streams it to a host PC via UART at 115200 baud. A Python script on the host reconstructs the image using IDCT and dequantisation.

System Architecture

Block Design Overview

The Vivado 2022.2 block design targets the ZedBoard (xc7z020clg484-1) and consists of four IP blocks connected via the AXI4-Lite interconnect, as shown in Figure 1.

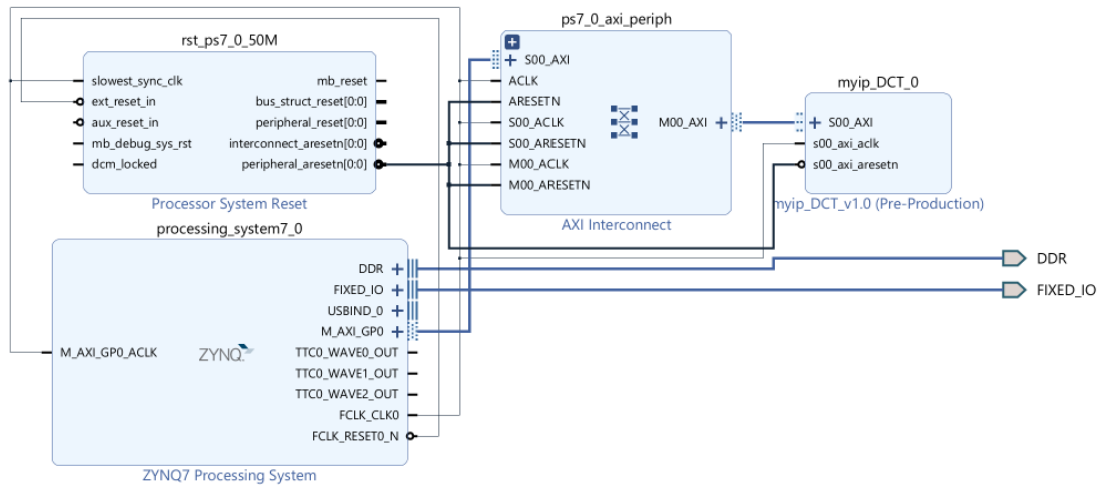


Figure 1: Vivado Block Design – Zynq PS + AXI Interconnect + myip_DCT_0

Table 1: Block Design IP Summary

IP Instance	VLNV	Function
processing_system7_0	xilinx.com:ip:processing_system7:5.5	ARM Cortex-A9 PS
ps7_0_axi_periph	xilinx.com:ip:axi_interconnect:2.1	AXI Interconnect
rst_ps7_0_50M	xilinx.com:ip:proc_sys_reset:5.0	System Reset
myip_DCT_0	xilinx.com:user:myip_DCT:1.0	Custom DCT IP

System Data Flow

1. PS reads the 128×128 greyscale image from DDR memory block by block.
2. Each 8×8 pixel block (64 pixels, signed 16-bit after level shift) is written to the IP core via AXI registers.
3. PS pulses the **start** bit; the hardware pipeline executes autonomously.
4. PS polls the **STATE** register until **DONE** (state = 4).
5. PS reads back the RLE entries and count via AXI registers.
6. All 256 blocks are transmitted over UART to the host PC.
7. The Python script decodes RLE, dequantises, and applies IDCT to reconstruct the image.

IP Core Design

AXI4-Lite Register Map

Table 2: myip_DCT AXI4-Lite Register Map (Base Address: 0x43C00000)

Offset	Access	Name	Description
0x00	R/W	REG_CTRL	[0]=start pulse, [1]=pixel_write_en
0x04	R/W	REG_INDEX	[6:0] pixel write / RLE read index (0–64)
0x08	R/W	REG_PIXEL	[15:0] signed 16-bit input pixel
0x0C	R	REG_STATE	[2:0] pipeline state (0=IDLE to 4=DONE)
0x10	R	REG_RLE	[21:16]=run length, [15:0]=signed value
0x14	R	REG_COUNT	[6:0] total RLE entries including EOB

Hardware Pipeline State Machine

The IP core implements a 5-state FSM as illustrated below:

Table 3: FSM State Descriptions

State	Name	Operation
0	IDLE	Waits for rising edge on start ; accepts pixel writes
1	STAGE1	Row-wise 1-D DCT: $T[r][c] = \sum_k C[r][k] \cdot P[k][c]$; 64 cycles
2	STAGE2	Column-wise 1-D DCT + $\gg 14$ descale; 64 cycles
3	QZ_RLE	Zigzag fetch, reciprocal quantisation, JPEG RLE encode; ~66 cycles
4	DONE	RLE output stable; waits for PS to deassert start

DCT Implementation

The 2-D DCT is factored into two separable 1-D passes using the integer-scaled coefficient matrix C where $C[0][k] = 64$ (DC row) and $C[u][k] \approx \text{round}(90 \cdot \cos(\pi u(2k + 1)/16))$ for $u = 1 \dots 7$. The combined transform is:

$$\text{DCT}[r][c] = \frac{1}{2^{14}} \sum_{k=0}^7 \left(\sum_{j=0}^7 C[r][j] \cdot P[j][k] \right) C[c][k]$$

Quantisation

Quantisation uses pre-computed reciprocals of the JPEG standard luminance Q-table at quality 50. Each coefficient $D[i]$ is quantised as:

$$Q_z[i] = \left\lfloor \frac{D[ZZ(i)] \cdot RQ[i] + 2^{19}}{2^{20}} \right\rfloor$$

where $RQ[i] = \text{round}(2^{20}/Q_{\text{LUMA}}[ZZ(i)])$. This avoids hardware division entirely.

RLE Encoding

After quantisation in zigzag order, JPEG-compatible RLE is applied:

- **DC coefficient:** always emitted as (0, val).
- **AC non-zero:** emitted as (zero_run, val).
- **ZRL:** run of 16 zeros emitted as (15, 0).
- **EOB:** end-of-block marker (0, 0) after last non-zero AC.

Design Code

Verilog – AXI4-Lite Slave and DCT Engine (myip_DCT_v1_0_S00_AXI)

```

1  `timescale 1 ns / 1 ps
2
3  // Register map:
4  // 0x00 slv_reg0 [0]=start [1]=pixel_write_en
5  // 0x04 slv_reg1 [6:0]=index
6  // 0x08 slv_reg2 [15:0]=signed pixel value
7  // 0x0C (read) {29'b0, state[2:0]}
8  // 0x10 (read) {10'b0, rle_mem[index]} = {10'b0, run[5:0], val
   [15:0]}
9  // 0x14 (read) {25'b0, rle_count[6:0]}
10
11 module myip_DCT_v1_0_S00_AXI #(
12     parameter integer C_S_AXI_DATA_WIDTH = 32,
13     parameter integer C_S_AXI_ADDR_WIDTH = 6
14 ) (
15     input wire S_AXI_ACLK, S_AXI_ARESETN,
16     // ... AXI4-Lite ports ...
17 );
18
19 // DCT coefficient matrix (integer-scaled)
20 localparam signed [15:0]
21     C00=16'sd64, C10=16'sd89, C20=16'sd83, C30=16'sd75,
22     C40=16'sd64, C50=16'sd50, C60=16'sd35, C70=16'sd18;
23 // ... full 8x8 matrix ...
24
25 // Pipeline memories
26 reg signed [15:0] pixel_mem [0:63]; // 8x8 input pixels
27 reg signed [47:0] temp_mem [0:63]; // Stage 1 intermediate
28 reg signed [31:0] dct_mem [0:63]; // Stage 2 DCT output
29 reg [21:0] rle_mem [0:64]; // RLE output {run[5:0], val[15:0]}
30 reg [6:0] rle_count;
31
32 // Quantisation: combinational reciprocal multiply

```

```

33 wire signed [52:0] qz_prod_w =
34     $signed(dct_mem[zigzag_rom(qz_i[5:0])) *
35     $signed({1'b0, recip_rom(qz_i[5:0])});
36 wire signed [52:0] qz_rounded_w = qz_prod_w + 53'sd524288; // +219
37 wire signed [52:0] qz_shifted_w = qz_rounded_w >>> 20;
38
39 // FSM: IDLE -> STAGE1 -> STAGE2 -> QZ_RLE -> DONE
40 always @(posedge S_AXI_ACLK) begin
41     case (state)
42     3'd0: if (slv_reg0[0] && !start_r) begin // rising edge on start
43         state <= 3'd1; row <= 0; k <= 0;
44     end
45     3'd1: begin // STAGE1: row-wise DCT
46         // acc[c] += C[row][k] * pixel[k][c]
47         if (k == 7) begin
48             temp_mem[row*8+0] <= acc[0] + ...;
49             if (row == 7) state <= 3'd2;
50             else row <= row + 1;
51         end else k <= k + 1;
52     end
53     3'd2: begin // STAGE2: column-wise DCT + >>14
54         if (k == 7) begin
55             dct_mem[row*8+c] <= (acc[c] + ...) >>> 14;
56             if (row == 7) state <= 3'd3;
57         end
58     end
59     3'd3: begin // QZ_RLE
60         // Zigzag fetch + saturate to 16-bit
61         if (qz_i <= 63) begin
62             qz_val_r <= (qz_shifted_w > 32767) ? 16'sd32767 :
63                 (qz_shifted_w < -32768) ? -16'sd32768 :
64                 qz_shifted_w[15:0];
65             qz_valid_r <= 1'b1;
66             qz_i <= qz_i + 1;
67         end
68         // RLE consume: DC always emitted; AC zeros counted; EOB
69         // at end
70         if (rle_i == 64) begin
71             rle_mem[last_nz_out_idx] <= {6'd0, 16'd0}; // EOB
72             rle_count <= last_nz_out_idx + 1;
73             state <= 3'd4;
74         end
75     end
76     3'd4: if (!slv_reg0[0]) state <= 3'd0; // wait for deassert
77 endcase
78 end
endmodule

```

Listing 1: AXI4-Lite Interface and DCT Pipeline (key excerpts)

C – Vitis Application (main.c)

```

1 #include "xil_printf.h"
2 #include <xparameters.h>
3 #include <stdint.h>
4 #include "image_data.h" /* NUM_BLOCKS, image_blocks[][] */
5
6 #define DCT_BASE          0x43C00000

```

```

7  #define REG_CTRL      0x00  /* [0]=start [1]=pixel_write_en */
8  #define REG_INDEX     0x04  /* [6:0] index */
9  #define REG_PIXEL     0x08  /* [15:0] pixel */
10 #define REG_STATE     0x0C  /* [2:0] state */
11 #define REG_RLE_DATA  0x10  /* [21:16]=run [15:0]=val */
12 #define REG_RLE_COUNT 0x14  /* [6:0] count */
13
14 #define RD(off)      (*(volatile uint32_t*)(DCT_BASE + (off)))
15 #define WR(off,v)    (*(volatile uint32_t*)(DCT_BASE + (off)) = (uint32_t)(
    v))
16
17 typedef struct { uint8_t run; int16_t val; } rle_t;
18
19 static int hw_run_block(const int16_t pix[64],
20                        rle_t *rle, uint32_t *cnt)
21 {
22     /* Load 64 pixels into pixel RAM */
23     for (int i = 0; i < 64; i++) {
24         WR(REG_INDEX, (uint32_t)i);
25         WR(REG_PIXEL, (uint32_t)(uint16_t)pix[i]);
26         WR(REG_CTRL, 2u); /* assert pixel_write_en */
27         WR(REG_CTRL, 0u); /* deassert -- creates write pulse */
28     }
29     WR(REG_CTRL, 1u); /* rising edge on start */
30
31     /* Poll until DONE (state == 4) */
32     uint32_t timeout = 2000000u;
33     while ((RD(REG_STATE) & 0x7u) != 4u)
34         if (--timeout == 0u) return -1;
35
36     WR(REG_CTRL, 0u); /* deassert start */
37
38     /* Read RLE results */
39     *cnt = RD(REG_RLE_COUNT) & 0x7Fu;
40     for (uint32_t i = 0; i < *cnt; i++) {
41         WR(REG_INDEX, i);
42         uint32_t raw = RD(REG_RLE_DATA);
43         rle[i].run = (uint8_t)((raw >> 16) & 0x3Fu);
44         rle[i].val = (int16_t)(raw & 0xFFFFu);
45     }
46     return 0;
47 }
48
49 int main(void)
50 {
51     xil_printf("START\r\n");
52     xil_printf("%sIMAGE_DIMS %d %d\r\n", IMG_ROWS, IMG_COLS);
53     /* Send QTABLE and ZIGZAG header lines ... */
54
55     for (int blk = 0; blk < NUM_BLOCKS; blk++) {
56         const int16_t *pix = image_blocks[blk];
57         rle_t hw_rle[65];
58         uint32_t hw_cnt = 0;
59         int ret = hw_run_block(pix, hw_rle, &hw_cnt);
60         if (ret != 0) { xil_printf("ERROR BLOCK %d\r\n", blk); continue;
61             }
62         xil_printf("%sBLOCK %d", blk);

```

```

63     for (uint32_t i = 0; i < hw_cnt; i++)
64         xil_printf(" %d %d", (int)hw_rle[i].run, (int)hw_rle[i].val)
65         ;
66     xil_printf("\r\n");
67 }
68 xil_printf("END\r\n");
69 while(1);
70 return 0;
71 }

```

Listing 2: PS Application – Pixel Write, Start, Poll, and RLE Read

Python – UART Receiver and Image Reconstruction (receive.py)

```

1  import serial, numpy as np, cv2, pickle, matplotlib.pyplot as plt
2
3  PORT = "COM9"; BAUD = 115200
4  ser = serial.Serial(PORT, BAUD, timeout=1)
5
6  # --- Receive QTABLE, ZIGZAG, and all RLE BLOCK lines over UART ---
7  QTABLE, ZZ, blocks = [], [], []
8  while True:
9      line = ser.readline().decode(errors='ignore').strip()
10     if line == "START": continue
11     elif line.startswith("%IMAGE_DIMS"):
12         IMG_ROWS, IMG_COLS = int(line.split()[1]), int(line.split()[2])
13     elif line.startswith("%QTABLE"):
14         QTABLE = list(map(int, line.split()[1:]))
15     elif line.startswith("%ZIGZAG"):
16         ZZ = list(map(int, line.split()[1:]))
17     elif line.startswith("%BLOCK"):
18         tokens = line.split()
19         blk_id = int(tokens[1])
20         data = list(map(int, tokens[2:]))
21         blocks.append((blk_id, data))
22     elif line == "END": break
23 ser.close()
24
25 # --- Reconstruction ---
26 C_hw = np.array([[64,64,64,64,64,64,64,64],
27                  [89,75,50,18,-18,-50,-75,-89],
28                  [83,35,-35,-83,-83,-35,35,83],
29                  [75,-18,-89,-50,50,89,18,-75],
30                  [64,-64,-64,64,64,-64,-64,64],
31                  [50,-89,18,75,-75,-18,89,-50],
32                  [35,-83,83,-35,-35,83,-83,35],
33                  [18,-50,75,-89,89,-75,50,-18]], dtype=np.float64)
34 Q_FLAT = np.array(QTABLE, dtype=np.float64).flatten()
35 img = np.zeros((IMG_ROWS, IMG_COLS), dtype=np.uint8)
36
37 for blk_id, data in blocks:
38     # RLE decode
39     qz = np.zeros(64); pos = 0
40     for i in range(0, len(data)-1, 2):
41         run, val = data[i], data[i+1]
42         if i == 0: qz[0] = val; pos = 1; continue
43         if run == 0 and val == 0: break
44         pos += run

```

```

45         if pos < 64: qz[pos] = val; pos += 1
46     # Dequantise
47     dct = np.zeros(64)
48     for i in range(64): dct[ZZ[i]] = qz[i] * Q_FLAT[ZZ[i]]
49     # Inverse DCT (exact hardware-matched)
50     D = dct.reshape((8,8)).T           # transpose to match MATLAB layout
51     P = C_hw.T @ D @ C_hw / 65536.0 + 128
52     P = np.round(P).clip(0,255).astype(np.uint8)
53     # Place block
54     br, bc = blk_id // 16, blk_id % 16
55     img[br*8:br*8+8, bc*8:bc*8+8] = P
56
57 cv2.imwrite("reconstructed.png", img)
58 plt.imshow(img, cmap='gray'); plt.title("Reconstructed"); plt.show()

```

Listing 3: Host Python Script – UART Reception and IDCT Reconstruction

Procedure

1. Create a Vivado 2022.2 project targeting the ZedBoard (xc7z020clg484-1).
2. Use **Tools** → **Create and Package New IP** to create the custom AXI4-Lite peripheral; edit the Verilog template to implement the DCT + quantisation + RLE engine. Package and add to the local IP repository.
3. Create a block design; add the Zynq7 PS, AXI Interconnect, Processor System Reset, and the custom `myip_DCT` IP. Run connection automation and validate.
4. Right-click the block design and **Create HDL Wrapper**.
5. Run Synthesis → Implementation → **Generate Bitstream**.
6. Export hardware (**File** → **Export** → **Export Hardware**, include bitstream).
7. Open Vitis 2022.2; create a new Application Project from the exported `.xsa`.
8. Add `main.c` and `image_data.h` to the `src/` directory. Verify `DCT_BASE = 0x43C00000` matches `xparameters.h`.
9. Build the project, program the FPGA, and run via **Launch on Hardware (GDB)**.
10. On the host PC, set the correct COM port in `receive.py` and run: `python receive.py`
11. Observe `reconstructed.png` for visual verification of compression quality.

Results

UART Terminal Output (Python Host)

Figure 2 shows the Python script output confirming receipt of metadata (image dimensions, Q-table, zigzag order) and all 256 RLE-encoded 8×8 blocks from the FPGA over UART.

```

$ python receive.py
Waiting for FPGA stream...
FPGA Transmission Started
%IMAGE_DIMS 128 128
Image Size: 128 x 128, Blocks/Row: 16
%QTABLE 16 11 10 16 24 40 51 61 ...
QTABLE Received
%ZIGZAG 0 1 8 16 9 2 3 10 17 24 ...
ZIGZAG Received
%BLOCK 0 0 61 0 -2 0 -1 0 -2 0 1 0 -3 3 -1 0 -1 3 -1 0 0
Received Block 0
%BLOCK 1 0 50 0 -1 0 -2 1 -1 0 4 3 -2 0 -1 9 -1 0 0
Received Block 1
%BLOCK 2 0 64 0 -11 0 -2 0 -2 0 -1 0 3 0 -4 ...
Received Block 2
... [Blocks 3 -- 253] ...
%BLOCK 254 0 105 0 -3 0 4 0 4 0 -2 0 -7 ...
Received Block 254
%BLOCK 255 0 142 0 -58 0 10 0 -5 0 -2 0 39 ...
Received Block 255
END
FPGA Transmission Complete
UART Reception Finished
Compressed file saved -> compressed_image.bin

Starting Reconstruction for 128x128 image ...
Reconstructed Block 0 -> (0,0)
Reconstructed Block 1 -> (0,8)
... [Blocks 2 -- 254] ...
Reconstructed Block 255 -> (120,120)
Reconstructed image saved as reconstructed.png

```

Figure 2: Python terminal output showing UART data reception from FPGA

Image Compression Output

The original 128×128 greyscale test image was processed as 256 blocks of 8×8 pixels by the hardware IP core. Figure 3 shows the original image alongside the reconstructed image after JPEG-like compression.



Figure 3: Left: Original 128×128 image. Right: Reconstructed after hardware DCT + quantisation (JPEG Q50) + RLE + software IDCT.

Compression Statistics

Table 4: Compression Performance Summary

Parameter	Value
Image size	128×128 pixels, 8-bit greyscale
Number of 8×8 blocks	256
Original uncompressed size	16,384 bytes
Total RLE entries (all blocks)	3,715
Quantisation table	JPEG Standard Luminance
AXI clock frequency	100 MHz (FCLK0)
Hardware cycles per block	≈200 clock cycles
UART baud rate	115,200 bps

Block-Level RLE Sample

The first few blocks received over UART demonstrate the sparse nature of the compressed data. For Block 0 (top-left 8×8 patch):

```
%BLOCK 0 0 61 0 -2 0 -1 0 -2 0 1 0 -3 3 -1 0 -1 3 -1 0 0
```

The DC coefficient is 61 (run=0, val=61). Subsequent AC entries show small non-zero values with zero-run prefixes. The trailing (0, 0) is the EOB marker, confirming only 9 significant RLE entries out of a possible 64 coefficients – a typical result for a relatively smooth image patch at JPEG quality 50.

Conclusion

A custom AXI4-Lite IP core implementing JPEG-like image compression was successfully designed in Verilog, integrated into a Zynq-7000 SoC block design, and verified on the ZedBoard. The hardware pipeline executes the full 2-D DCT, zigzag quantisation, and RLE encoding autonomously in approximately 200 clock cycles per 8×8 block at 100 MHz, demonstrating the advantage of offloading compute-intensive transforms to the Programmable Logic.

Key achievements:

- Complete AXI4-Lite slave interface with pixel write and RLE read ports.
- Two-stage separable 1-D DCT without hardware division, using integer-scaled coefficients and arithmetic right-shifts.
- Reciprocal-multiplication quantisation eliminating all dividers from the critical path.
- JPEG-compatible RLE with DC, AC, ZRL, and EOB encoding in a two-stage pipeline.
- End-to-end system demonstration: FPGA → UART → Python IDCT → PNG reconstruction.